

Lab 2: Assembly: Reverse Engineering & Implementation

CSCI 3334 · UTRGV Fall 2026

Overview

Weight: 10% | **Out:** Wed Sep 23 | **Due:** Wed Oct 7

You will reverse-engineer compiled x86-64 functions and defuse a binary bomb. Both parts are graded in levels. The Level 1 items in the two parts are worth 60 of the 100 points, so do them first.

Part 1 starts with an ungraded warmup that takes about ten minutes. In Part 2, only the first three bomb phases are required; the last three are extra credit.

Important

Checkpoint: Mon Oct 5

Show me Part 1 building with Level 1 passing: `make test P=1` and `make test P=2` both print `PASS`. That is two functions, each with one loop.

The checkpoint is graded pass/fail on effort, not correctness, and counts toward participation. It is there so you get help early if you are stuck.

Before you start: you need Linux x86-64

Important

This lab does not run on a Mac

The `mystery*.o` files are Linux x86-64 ELF objects, and the bomb is a Linux x86-64 executable. On macOS (Apple Silicon or Intel), `make` cannot link them and the bomb will not run. `gdb` is also not available on Apple Silicon.

Use one of these:

- WSL2 on Windows (set up in Lab 0)
- a Linux machine or VM
- the department lab machines

Labs 0 and 1 were plain C and ran anywhere. This one does not, so set up your environment on the first day.

Setup

Download `lab2.zip` from the Lab 2 assignment on **Brightspace** and unzip it inside your Linux environment. It unpacks to a `lab2/` folder that holds everything for both parts: the mystery objects, `solutions.c`, the tests, and the bomb.

```
unzip lab2.zip
cd lab2
make          # builds ./btest against the mystery objects
make test     # checks the warmup and all five
make test P=0 # checks only the warmup, mystery0
make test P=3 # checks only mystery3
make disas P=3 # disassembles mystery3.o
```

Unzip it with `unzip` in Linux, not with Windows Explorer. If `./bomb` says `Permission denied`, the execute permission was lost on the way; run `chmod +x bomb` to fix it.

Part 1: Reverse Engineering (60 pts + 10 bonus)

You get a warmup and five compiled functions. For each one, work out what it does and write a C function that behaves the same way.

Note

Start with the warmup, then Level 1

Almost everything you need was covered in Lectures 6 and 7, and the three instructions that were not are explained below. The warmup, `mystery0`, is two instructions with no loop, so you can practice the steps on something small. `mystery1` and `mystery2` have one loop each and are a good first evening's work. Each is worth 15 points, the same as the hardest required function.

Level 0: Warmup (0 points)

#	Function	Signature	What to look for
0	<code>mystery0</code>	<code>int (int, int)</code>	two <code>lea</code> instructions, no loop, no branch

Level 1: One loop

#	Function	Signature	What to look for
1	<code>mystery1</code>	<code>int (int)</code>	one loop, one accumulator, one counter
2	<code>mystery2</code>	<code>int (unsigned)</code>	a shift and a mask: a loop over bits

Level 2: Arrays and arithmetic

#	Function	Signature	What to look for
3	<code>mystery3</code>	<code>int (const int *, int)</code>	scaled indexing (<code>%rdi,%rcx,4</code>), a <code>cmov</code> , two early returns
4	<code>mystery4</code>	<code>int (int, int)</code>	<code>cld + idivl</code> : which half of the result is kept?

Level 3: Branchless

#	Function	Signature	What to look for
5	<code>mystery5</code>	<code>int (const char *)</code>	one <code>cmpb</code> and an <code>adcl</code> that together replace two comparisons

Disassemble a function with:

```
objdump -d mystery1.o
```

Write your C version in `solutions.c`, then check it:

```
make test P=1    # just mystery1
make test        # all five
```

Warning

Name your functions `my_mysteryN`, not `mysteryN`

`mystery1` is already defined inside `mystery1.o`; that is the reference your answer is compared against. If you define another `mystery1`, the link fails with a duplicate symbol. The stubs in `solutions.c` already have the right names and signatures. Fill in the bodies and do not rename them.

Worked example

This function is not part of the lab. It shows the steps and what a finished answer looks like.

```
example:
    cml $1, %edi    # n <= 1?  (computes edi - 1)
    jle .one
    movl $1, %eax    # result = 1
.loop:
    imull %edi, %eax # result *= n
    subl $1, %edi    # n--
    cml $1, %edi
    jne .loop        # keep going while n != 1
    ret
.one:
    movl $1, %eax
    ret
```

Read it using the steps from Lectures 6 and 7:

1. **Arguments.** One int arrives in `%edi`. The result goes in `%eax`.
2. **Find the loop.** The backward `jne .loop` marks it.
3. **Name the registers.** `%eax` is the running product. `%edi` counts down.
4. **Read the guard.** `cml $1, %edi` followed by `jle` handles $n \leq 1$ before the loop.
5. **Describe it in one sentence.** “Multiply together every integer from n down to 2.” That is $n!$.

Tracing it, one instruction at a time

Reading the code gives you a guess. Tracing it checks the guess. Pick a small input (here $n = 4$) and write down what each register holds after every instruction. Only two registers change, so the table needs two columns.

Step	Instruction	%edi (n)	%eax (result)	What happened
1	<code>cml \$1, %edi</code>	4	?	computes $4 - 1 = 3$ and sets the flags; nothing is stored
2	<code>jle .one</code>	4	?	$4 \leq 1$ is false, so the jump is not taken
3	<code>movl \$1, %eax</code>	4	1	<code>result = 1</code>
4	<code>imull %edi, %eax</code>	4	4	<code>result = 1 × 4</code>
5	<code>subl \$1, %edi</code>	3	4	<code>n--</code>
6	<code>cml \$1, %edi / jne .loop</code>	3	4	$3 \neq 1$, so the jump is taken, back to step 4
7	<code>imull %edi, %eax</code>	3	12	<code>result = 4 × 3</code>
8	<code>subl \$1, %edi</code>	2	12	<code>n--</code>
9	<code>cml \$1, %edi / jne .loop</code>	2	12	$2 \neq 1$, taken
10	<code>imull %edi, %eax</code>	2	24	<code>result = 12 × 2</code>
11	<code>subl \$1, %edi</code>	1	24	<code>n--</code>

Step	Instruction	%edi (n)	%eax (result)	What happened
12	<code>cmpl \$1, %edi / jne .loop</code>	1	24	1 = 1, not taken, so the loop ends
13	<code>ret</code>	1	24	returns %eax

The result is $24 = 4 \times 3 \times 2 \times 1 = 4!$, which matches the sentence in step 5. The ? in the first two rows means %eax still holds an old value that the function never uses.

Notes on the trace:

- `cmp` does not store a result. It only sets the flags, and the jump after it reads them. Treat each `cmp` and its jump as one step, as rows 6, 9 and 12 do.
- The backward jump is the loop. Each time it is taken, the loop runs again. The row where it is not taken is where the loop ends.
- Trace the edge case as well. Try `n = 1`: the `jle` in step 2 is taken, the code jumps to `.one`, and it returns 1. That is the `n <= 1` guard. Early returns like this are where most wrong answers come from.

Trace every mystery function this way on paper. When `make test` reports a **FAIL**, tracing the failing input is usually the quickest way to find the problem.

The answer you would write:

```
int my_example(int n) {
    int result = 1;
    while (n > 1) { result *= n; n--; }
    return result;
}
```

Your turn: the warmup

Now use the same steps on `mystery0`. It is worth no points, but `make test` checks it like the others, so it also confirms that your setup works.

```
objdump -d mystery0.o
```

You will see two `lea` instructions and a `ret`. Work through it in this order:

1. **Arguments.** `a` is in %edi and `b` is in %esi. The answer goes in %eax. Inside the parentheses, `objdump` shows the 64-bit names %rdi, %rsi and %rax; they are the same registers.
2. **Read `lea` as arithmetic.** `lea D(B,I,S)`, `dst` does not access memory. It computes `dst = B + I*S + D`. If `D` is missing, it is 0.
3. **One instruction at a time.** Write what %eax holds after the first `lea`, in terms of `a`. Then write what it holds after the second, in terms of `a` and `b`. Note that `-0x1` is `-1`.
4. **Check a value by hand.** Take `a = 2`, `b = 1` and compute the result from the instructions before you write any C.
5. **Write it and test it.** Fill in `my_mystery0` in `solutions.c` and run `make test P=0`. It should print **PASS**. A **FAIL** shows the input and both answers; trace that input through the two instructions.

The graded functions use the same steps, with loops and branches added. They get harder in order, so do them in order.

Three instructions the lectures did not cover

Everything else in these five functions is from Lecture 6 or 7. These three are not, so here is what they do.

Instruction	What it does
<code>cld</code>	Sign-extends <code>%eax</code> into the 64-bit pair <code>%edx:%eax</code> . It always comes right before <code>idivl</code> , which needs a 64-bit dividend. It sets up the division and does no arithmetic of its own. (Intel syntax calls it <code>cdq</code> .)
<code>idivl r</code>	Signed divide of <code>%edx:%eax</code> by <code>r</code> . The quotient goes in <code>%eax</code> and the remainder in <code>%edx</code>. Work out which one the function keeps.
<code>adcl s, d</code>	<code>d = d + s + CF</code> : an add that also adds the carry flag. <code>adcl \$0, %edx</code> adds 1 when CF is set and 0 when it is not, so it counts a comparison's result without a branch.

You have also seen `movslq %edx, %rcx` in another form. It is the sign-extending move from Lecture 6, going from 32 to 64 bits, and it widens an `int` index so it can be used in an address.

Part 2: Bomb Lab (30 pts + 15 bonus)

You are given a 64-bit program called `bomb`. It reads input from `stdin` and “explodes” (calls `explode_bomb`) if the input is wrong. Your job is to find the input that defuses each phase.

Everyone has the same bomb. What is being assessed is whether you can read disassembly and use a debugger, and the Unit 2 exam questions are based on this lab.

You are encouraged to discuss technique: how to set a breakpoint, how to read a jump table, what `x/6dw $rsp` shows. Sharing a finished `answers.txt` is plagiarism under the syllabus policy. It will also leave you unprepared for the exam.

Level 1: required

Phase	Worth	What it involves	Where to start
1	10	a string comparison	the string is stored in the binary: try <code>strings bomb</code> , or <code>x/s</code> on the string it compares against
2	10	six numbers following a rule	find the first number, then the rule that produces the next one
3	10	a jump table	the first number picks a case and the second must match it; more than one answer works. Hard Exercise 5 in the Lab 2 Extra Practice Problems packet is a jump table of the same shape

Level 2: bonus

Phase	Worth	What it involves	Where to start
4	+5	recursion	a range check happens before the recursion; get inside the range first
5	+5	a character-indexed lookup table	only part of each character is used as the index

Phase	Worth	What it involves	Where to start
6	+5	ordering six items	it wants a permutation of 1–6; the no-repeat check runs before the ordering check

Phases 1–3 are the required part of the bomb. Defusing those three gives you full marks for Part 2. Phases 4–6 are extra credit, +5 each.

The bomb still asks for all six phases. If `answers.txt` has only three lines, the bomb prints **Phase 3 defused**, then **Error: premature EOF**, and stops. That is not an explosion and does not cost you anything. A three-line `answers.txt` is a complete submission.

Running it

```
./bomb                # type answers in, one line per phase
./bomb answers.txt    # or read them from a file (recommended)
```

Keep your answers in a file from the start. If you type them in, you have to retype all of them every time the bomb explodes. With a file you just run it again.

Tools

```
gdb ./bomb
(gdb) break explode_bomb    # do this first, every session
(gdb) run answers.txt
(gdb) disassemble          # where am I?
(gdb) stepi                # one instruction
(gdb) info registers
(gdb) x/s $rdi              # string at an address
(gdb) x/6dw $rsp            # six decimal words on the stack
```

Note

`gdb` and `objdump` both print AT&T syntax by default, which is what the lectures and CS:APP use. If you see output with no % signs, something has been set to Intel syntax.

Note

Two things at the top of every phase that you can skip

Every function in the bomb starts with `endbr64`. It is a marker for a CPU security feature and does nothing to the registers or memory, so read past it.

Most phases also start with `movq %fs:0x28, %rax` and end with a call to `__stack_chk_fail`. That is the stack canary from Lectures 7 and 8. The lectures showed the macOS form, which loads the value from `__stack_chk_guard`; on Linux it is read from `%fs:0x28`. It guards the stack and has nothing to do with the answer.

Tip

Set a breakpoint on `explode_bomb` first

The bomb keeps its function names, so `break explode_bomb` works as written, and so do commands like `disassemble phase_1`. When the program stops at the breakpoint, `backtrace` shows which phase called it. Without the breakpoint, a wrong answer ends the program before you can look at anything.

Submit `answers.txt` with one line per phase, in order. If you stopped at phase 3, it has three lines.

Grading

Criterion	Points
Part 1: <code>mystery0</code> , the warmup	0
Part 1: <code>mystery1</code> – <code>mystery4</code> (15 each)	60
Part 2: bomb phases 1–3 (10 each)	30
Comments in <code>solutions.c</code> explaining how you read each one	10
<i>Bonus</i> : <code>mystery5</code> , the branchless one	+10
<i>Bonus</i> : bomb phases 4–6 (5 each)	+15
<i>Bonus</i> : demo, walking me through your reasoning	+10

`mystery5` and bomb phases 4–6 are bonuses. You can score 100 without any of them. Bonus points are not capped, so a lab score above 100 is possible (up to 135).

Each mystery function is graded all-or-nothing by `make test`, so run it before you submit. A function that is almost right gets the same score as one that is completely wrong.

Demo: worth +10

Come to office hours (EIEAB 3.237, Mon/Wed 2:20–3:20, or by appointment) any time before **Wed Oct 7** and explain what you worked out. I will pick one mystery function or one bomb phase and ask how you solved it.

The demo is graded on your reasoning, not on how much you finished. For example: “I found the backward jump, saw that `%eax` only gets multiplied and `%edi` only goes down by one, and traced `n = 3` by hand to get 6, so it is a factorial.” That earns the bonus. Having all five correct but not being able to explain how you got them earns nothing.

Bring whatever you have. A half-finished `solutions.c` is fine for a demo, and it is a good way to get help with the part you are stuck on.

Hints

- AT&T syntax is `movq src, dst`: the destination is last.
- `cmpq b, a` computes `a - b`, the second operand minus the first. So after `cmpq %rsi, %rdi`, `jl` is taken when `rdi < rsi` (Lecture 7).
- `testq a, b` computes `a & b` and sets ZF and SF. `testq %rax, %rax` is the usual way to check whether a value is zero or NULL.
- Signed comparisons use `jl/jle/jg/jge`; unsigned comparisons use `jb/jbe/ja/jae`. The jumps you see tell you whether the C type is signed.
- `leaq` does arithmetic and does not access memory: `leaq (%rdi,%rdi,4), %rax` is `rax = rdi * 5` (Lecture 6).
- Arguments arrive in `%rdi, %rsi, %rdx, %rcx, %r8, %r9`; the result goes in `%rax`.
- The stack must be 16-byte aligned before a `call`.

Per function

- **mystery0**: the warmup, with no loop and no branch. Read the two `leas` in order as arithmetic. `(%rdi,%rdi,2)` is `a + a*2`.
- **mystery1**: find the loop. Which register accumulates, which one counts, and what is the exit condition? Look for a constant added to the bound before the loop starts.
- **mystery2**: a shift plus a mask in each iteration is a loop over bits. Note that it uses `shr`, not `sar`.
- **mystery3**: the scale in `(%rdi,%rcx,4)` tells you the element size. There are two early returns before the loop, and one of them does not return 0. The `cmov` uses a strict comparison, which decides what happens on a tie.
- **mystery4**: `cld` followed by `idivl` is signed division; the quotient is in `%eax` and the remainder in `%edx`. Both arguments are also changed before the loop. Test with negative inputs.
- **mystery5**: convert the constants to ASCII characters first. One `cmpb` plus `adcl` does a range test on both ends; work out why subtracting first means one unsigned compare is enough.

When you are stuck

`make test P=1` runs one function and prints the exact input that failed. Trace that input through the disassembly by hand. This is usually faster than rereading your C.

Submission

Upload `solutions.c` and `answers.txt` to the Lab assignment on **Brightspace**. Upload the files themselves, not a screenshot, a zip, or a link to a repo.

`answers.txt` has one line per bomb phase you defused, in order.

Your `solutions.c` must compile with the flags in the Makefile. Code that does not compile gets no correctness points, so build it before you upload.

References

- CS:APP §3.1–3.7
- x86-64 Reference